

PROJECT 02 REPORT

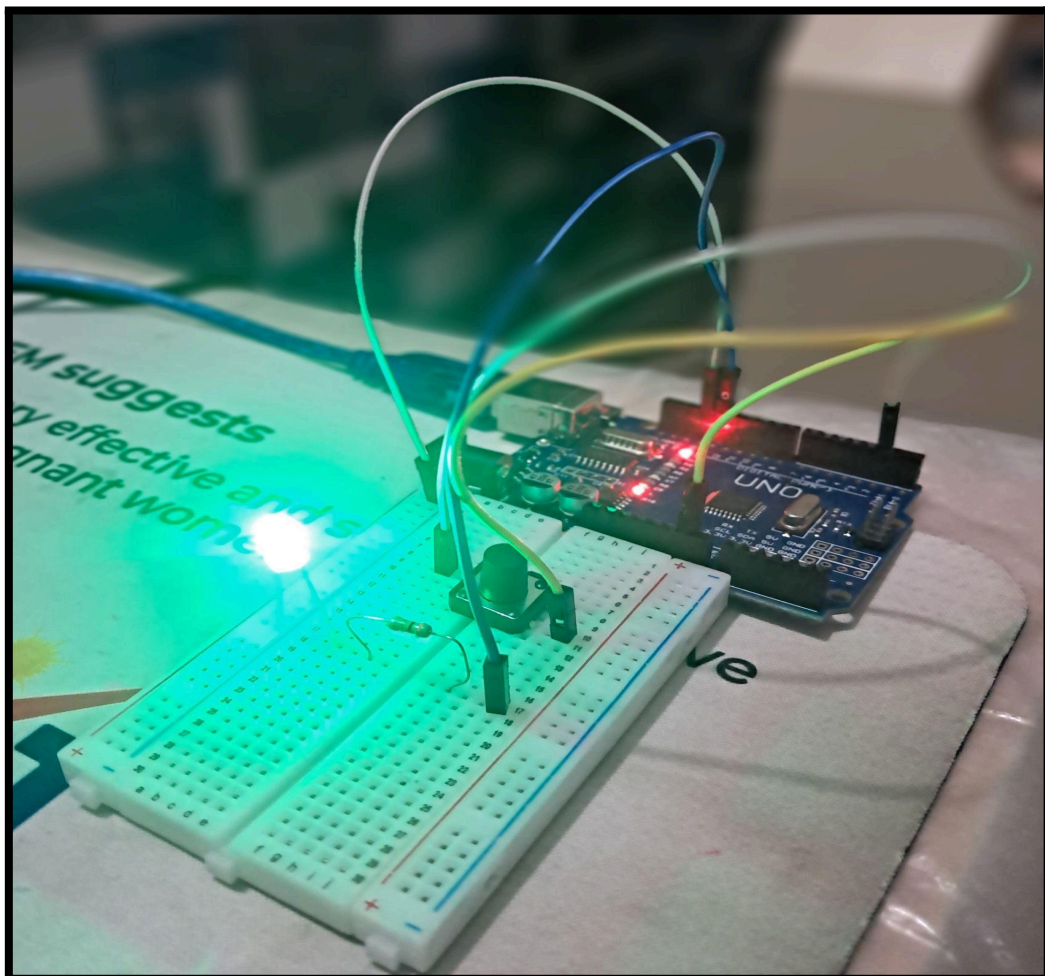
Push Button Controlled LED System

Name : Priyaansh Tiwari

Department : Electrical and Electronics Engineering

Date : June 2026

Duration : 3 days



1. Objective

The objective of this project was to learn how the Arduino receives digital inputs from external devices and makes decisions based on those inputs.

Unlike Project 1, which focused only on outputs, this project introduces user interaction through push buttons and concepts like digital input reading, pull-up resistors, edge detection, toggle logic, state machines, and switch debouncing

2. Components Used

Component	Quantity
Arduino Uno R3	1
Breadboard	1
Push Button	1
LEDs (Red,Yellow,Green)	Multiple
Resistors(300Ω)	1
Jumper Wires	Multiple
USB Cable	1

3. Software Used

- Arduino IDE
- Serial Monitor
- Windows Laptop

4. Theory

A push button is a digital input device that can exist in two states:

- Pressed
- Released

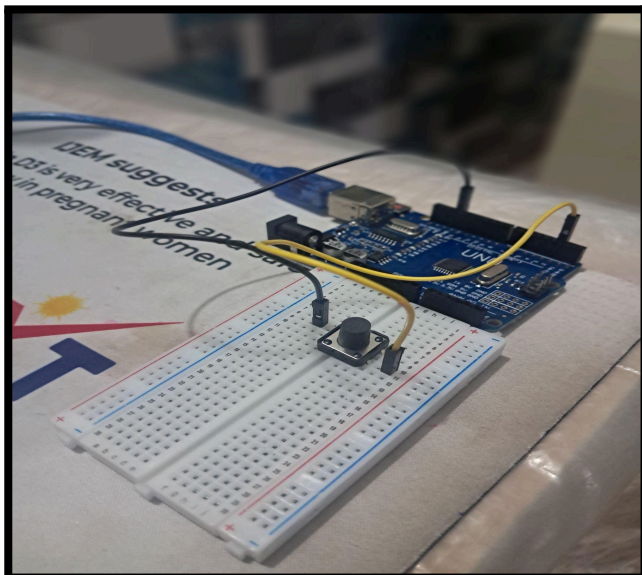
Arduino reads these states as:

- LOW (0)
- HIGH (1)

This project uses Arduino's internal pull-up resistor through: `pinMode(2, INPUT_PULLUP);` which eliminates the need for an external pull-up resistor and prevents floating inputs

5. Tasks Performed

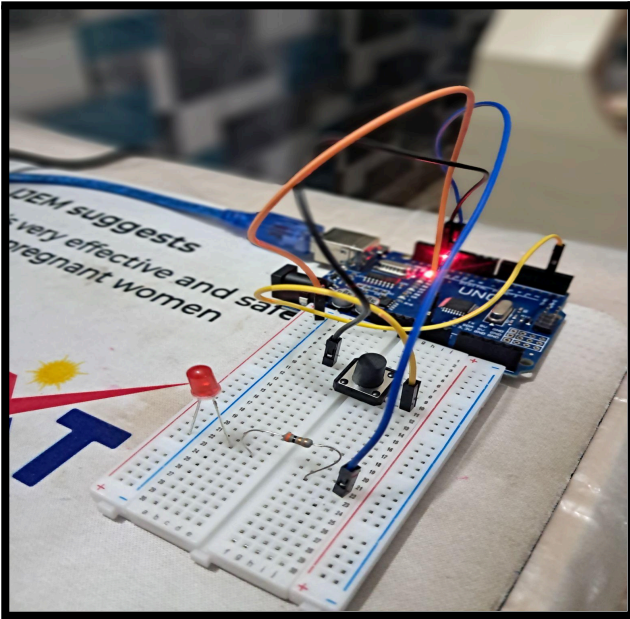
➤ Button State Reading



Video

A push button was interfaced with Arduino and the button state was monitored through the Serial Monitor and verified to produce HIGH when released and LOW when pressed.

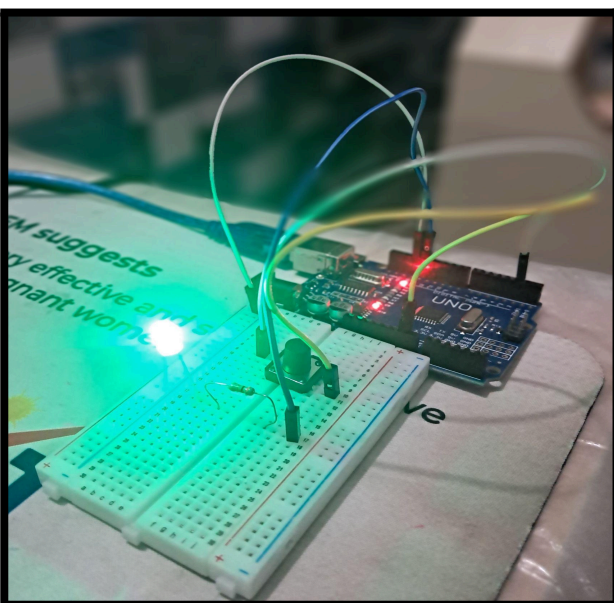
➤ Button Controlled LED



Video

An LED was controlled using push button input. The LED remained ON while the button was pressed and turned OFF as it was released.

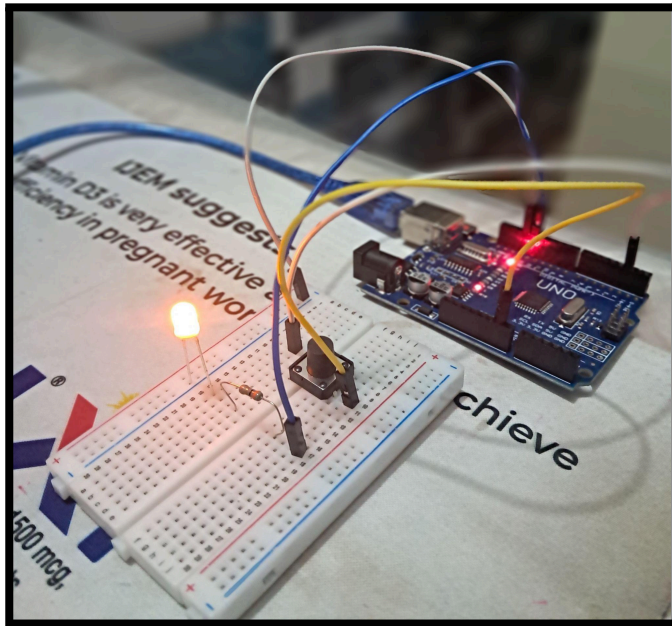
➤ Toggle LED System



Video

A toggle mechanism was implemented where each button press changed the LED state. The system used state variables and edge detection to remember the previous LED condition.

➤ **Multi-Mode LED Controller**



Video

A mode based controller was created using a single push button. The implemented modes were:

- ❖ Mode 0 – LED OFF
- ❖ Mode 1 – LED ON
- ❖ Mode 2 – LED BLINK

The mode variable was incremented on each valid button press to cycle through the modes.

➤ **Debugging and Debouncing**

While testing the toggle and multi-mode implementations, an issue was observed where a single button when pressed

occasionally triggered multiple actions. This resulted in skipping of modes and inconsistent behavior.

The cause was mechanical switch bouncing, where the button contacts rapidly make and break connection for a short period during a press.

A basic software debounce technique was implemented by introducing a small delay after detecting a valid button press which reduced the unintentional triggering.

6. Learning Outcomes

The following concepts were learned in making of this project:

- Digital Inputs
- `digitalRead()`
- `INPUT_PULLUP`
- Floating Inputs
- Serial Monitor Debugging
- Button Interfacing
- Toggle Logic
- Boolean Variables
- State Variables
- Current State and Previous State Tracking
- Edge Detection
- State Machines
- Switch Bounce
- Software Debouncing

7. Problems Faced

- I. Button initially remained at logic HIGH regardless of pressing due to incorrect breadboard placement of push button. **(The push button was repositioned correctly across the breadboard center gap)**
- II. Unexpected multiple mode changes occurred from a single press and the reason for this was switch bounce. **(Software debounce delay was implemented)**

8. Observations

- INPUT_PULLUP simplifies button wiring.
- Released state is read as HIGH(1) .
- Pressed state is read as LOW(0) .
- State variables allow systems to remember information.
- Edge detection enables detection of new button presses.
- State machines help organize multiple operating modes.
- Mechanical buttons do not produce perfectly clean signals.
- Debouncing is essential for reliable operation.

9. Applications and Future Scope

The concepts learned in this project forms the foundation of many embedded systems that require user interaction. Push button inputs, state based control and mode selection are commonly used in home appliances, industrial control panels, and consumer electronic devices.

The project can be further enhanced by incorporating additional features such as countdown timer displays, IoT-based monitoring and control using ESP32, and custom PCB implementation for a compact and professional hardware design.

10. Conclusion

This project successfully introduced digital input handling and decision-making in embedded systems. The project marks an important transition from simple output control to interactive embedded systems .